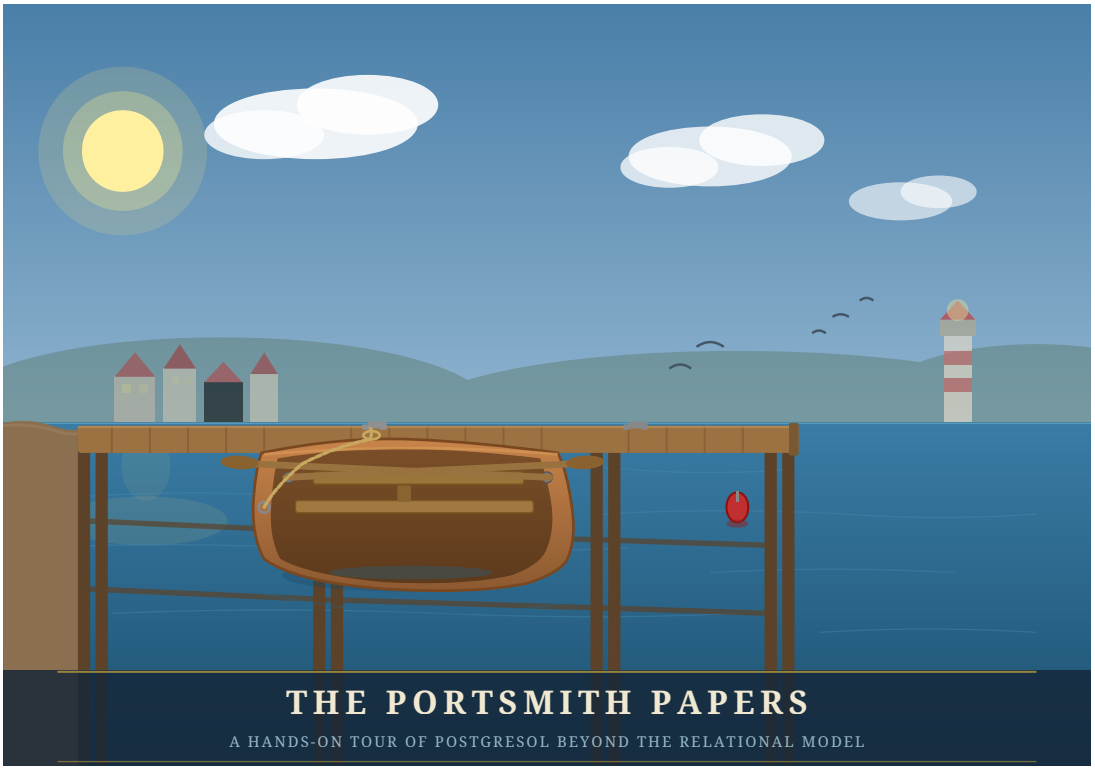
The Portsmith Papers: A Hands-On Tour of PostgreSQL Beyond the Relational Model

The Portsmith Papers

A Hands-On Tour of PostgreSQL Beyond the Relational Model



> *"PostgreSQL is not a database with extensions.*
> *It is an extensible data platform that happens to speak SQL."*

About This Book

Most PostgreSQL tutorials end where the interesting work begins.

Once you know how to `SELECT`, `JOIN`, and `GROUP BY`, you have mastered perhaps twenty percent of what PostgreSQL can do. The remaining eighty percent — geospatial queries, semantic search, real-time notifications, fuzzy matching, vector embeddings, distributed coordination, and more — lives in a ecosystem of extensions, index types, and language features that most practitioners never discover.

This book is a guided tour of that other eighty percent.

Each chapter is built around a concrete engineering problem faced by the fictional city of **Portsmith** and its data platform team. We store business directories as semi-structured JSON documents. We route emergency services using geospatial proximity queries. We build a job queue with no message broker. We search municipal records with fuzzy matching that survives typos and OCR errors. We expose the entire platform as a REST API with zero application code.

Every chapter follows the same structure: synthetic data is generated first, giving you a realistic dataset to work against, and then a series of exercises walks you from first principles to production-ready technique. The exercises are written to be done — not just read.

By the end, you will see PostgreSQL not as a place to store rows, but as a programmable data infrastructure layer capable of doing work that most teams reach for separate specialized systems to handle.

What you will need:

- A working PostgreSQL 16 installation (setup covered in Appendix A)
- Python 3.12+
- A Debian-based Linux environment
- Familiarity with basic SQL (`SELECT`, `INSERT`, `JOIN`, `GROUP BY`)
- Curiosity about what else is in there

Author

Chris Lee

Edition 1.0 — Portsmith, 2026

Chapter 1 — JSONB: Semi-Structured Data Without a Schema Tax

> *"A schema is a prediction about the future. JSONB lets you hedge."*

Background

Relational databases enforce a contract: every row in a table has exactly the same columns. That contract is usually a feature — it catches mistakes and makes queries predictable. But sometimes you genuinely don't know what columns you'll need until the data arrives. A business directory is a good example. A restaurant has hours, cuisine, and a reservations policy. A hardware shop has trade accounts and parking. A hotel has star ratings and room types. Forcing them all into the same set of columns means drowning in NULLs or building a rats' nest of one-to-many extension tables.

PostgreSQL's `JSONB` type lets you store arbitrary JSON documents in a column while keeping the rest of your row fully relational. It is not a NoSQL escape hatch — it is an indexed, queryable, patchable document field that sits inside a SQL table. You can `WHERE` on it, join against it, aggregate over it, and update individual keys without rewriting the whole document.

This chapter works through all the operators and patterns you'll reach for most often. The exercises are short. Run every one, then try a variation of your own.

The Scenario

Portsmith's city council maintains a public business directory. Every business gets a short relational record — an ID, a name, an address, and a neighbourhood — but the supplementary detail varies so widely by business type that a single fixed schema would be unworkable.

The `businesses` table therefore stores all of that variable metadata in a `details` JSONB column. Each business category contributes its own keys:

Category	JSONB keys present
`restaurant`	`cuisine`, `price_range`, `rating`, `hours`, `tags`, `accepts_reservations`, `outdoor_seating`
`retail`	`subcategory`, `rating`, `hours`, `tags`, `payment_methods`, `has_parking`
`service`	`subcategory`, `rating`, `hours`, `tags`, `appointment_required`, `specialties`
`accommodation`	`subcategory`, `star_rating`, `rating`, `hours`, `amenities`, `room_types`, `price_per_night`
`entertainment`	`subcategory`, `rating`, `hours`, `tags`, `live_music`, `age_restriction`

The `hours` value is itself a nested object keyed by day of week (`mon`–`sun`). Each day is either `null` (closed) or `{"open": "HH:MM", "close": "HH:MM"}`. Several businesses also carry optional keys that appear only for their category, like `social` (social media handles), `happy_hour`, or `approved_brands`.

This heterogeneity is intentional — it is what makes the data a good vehicle for learning JSONB operators.

Exercise Goals

By the end of this chapter you will be able to:

- Extract values from a JSONB document using `->`, `->>`, and `#>>`, and explain why the operator choice changes the result type.
- Filter rows using the containment operator `@>` and the key-existence operators `?`, `?|`, and `?&`.
- Create a GIN index on a JSONB column and confirm that PostgreSQL uses it.
- Update a document in place with `jsonb_set` and the `||` merge operator without rewriting the whole column.
- Expand a JSONB array into a set of rows with `jsonb_array_elements` and aggregate the results.
- Write `jsonb_path_query` expressions to filter on nested values.

Installation

1 — PostgreSQL

If PostgreSQL 16 is not already installed:

```
sudo apt update
sudo apt install -y postgresql-16 postgresql-client-16
```

Confirm the server is running:

```
pg_lsclusters
```

You should see a cluster at version 16 in the `online` state.

2 — Python 3.12 and psycopg

```
sudo apt install -y python3.12 python3.12-venv
```

Create a virtual environment in the book's working directory and install the PostgreSQL driver:

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install "psycopg[binary]"
```

> **Note:** `psycopg` (version 3) is the modern Python driver for PostgreSQL. It is not available as an `apt` package for Python 3.12, so we install it via pip into the virtual environment. The `[binary]` extra bundles a pre-compiled C extension so you don't need `libpq-dev`.

Loading the Data

Create the database

```
# Create a role matching your OS user (skip if it already exists)
sudo -u postgres createuser --createdb "$(whoami)"

# Create the portsmith database
createdb portsmith
```

Run the seed script

From the `book/` directory, with the virtual environment active:

```
python data/ch01_seed.py
```

Expected output:

```
Connecting to: dbname=portsmith
Creating schema …
Inserting 48 businesses …
Done — 48 rows in businesses.
```

Verify the load

Open a `psql` session:

```
psql portsmith
```

Run these checks. If all three pass, the data is correct.

Check 1 — Row count and column types:

```
\d businesses
```

```
                                    Table "public.businesses"
    Column     |  Type   | Collation | Nullable |
Default
---------------+---------+-----------+----------
+-----------------------------------
 id            | integer |           | not null |
nextval('businesses_id_seq'::regclass)
 name          | text    |           | not null |
 address       | text    |           | not null |
 neighbourhood | text    |           | not null |
 details       | jsonb   |           | not null |
```

Check 2 — Counts by neighbourhood:

```
SELECT neighbourhood, COUNT(*) AS businesses
FROM   businesses
GROUP  BY neighbourhood
ORDER  BY neighbourhood;
```

```
   neighbourhood    | businesses
--------------------+------------
 Harbour District   |          9
 Industrial Port    |          7
 Northgate          |          9
 Old Town           |          9
 Riverside          |          9
```

```
 University Quarter |          5
(6 rows)
```

Check 3 — Counts by category:

```sql
SELECT details->>'category' AS category, COUNT(*) AS businesses
FROM   businesses
GROUP  BY category
ORDER  BY category;
```

```
    category     | businesses
-----------------+------------
 accommodation   |          5
 entertainment   |          6
 restaurant      |         15
 retail          |         12
 service         |         10
(5 rows)
```

If all three match, proceed to the exercises.

Exercises

Exercise 1 — The Three Extraction Operators

JSONB has two families of accessor. The arrow operators (`->` and `->>`) navigate one key at a time. The path operator (`#>>`) jumps straight to a deeply nested value. The difference between them is the **type they return**.

1.1 — Arrow operators

Run these two queries and compare the result types:

```sql
-- -> returns JSONB
SELECT name,
       details -> 'rating' AS rating_jsonb
FROM   businesses
LIMIT  5;

-- ->> returns TEXT
SELECT name,
       details ->> 'rating' AS rating_text
FROM   businesses
LIMIT  5;
```

Look carefully at the column headings in `psql` — one will show `jsonb`, the other `text`. This distinction matters the moment you try to do arithmetic:

```
-- This works: cast text to numeric
SELECT name,
       (details ->> 'rating')::numeric AS rating
FROM   businesses
ORDER  BY rating DESC
LIMIT  5;
```

```
          name           | rating
-------------------------+--------
 Lighthouse Bookshop     |    4.9
 Finch & Sons Barbers    |    4.9
 Portsmith Vet. Clinic   |    4.8
 River Bend Bakery       |    4.8
 Quarter Note Jazz Club  |    4.8
(5 rows)
```

> **Key point:** Use `->` when you want to keep working with JSONB (e.g., to navigate deeper). Use `->>` when you need the final value as text for comparisons, casting, or display.

1.2 — The path operator `#>>`

Navigating nested keys with chained `->` quickly becomes unreadable. The path operator takes an array of keys and returns the leaf as text in one step.

```
-- Chained arrows (verbose)
SELECT name,
       details -> 'hours' -> 'fri' ->> 'close' AS friday_close
FROM   businesses
WHERE  details ->> 'category' = 'restaurant'
LIMIT  8;

-- Path operator (equivalent, cleaner)
SELECT name,
       details #>> '{hours,fri,close}' AS friday_close
FROM   businesses
WHERE  details ->> 'category' = 'restaurant'
LIMIT  8;
```

Both produce the same result. Notice that some rows return `NULL` — those restaurants are closed on Fridays (the `fri` key is JSON `null`).

```
        name          | friday_close
----------------------+--------------
 The Gilded Clam      | 22:30
```

```
 Anchor & Oar Tavern | 01:00
 Bella Napoli        | 23:00
 Le Petit Bistro     | 14:30
 Dragon Palace       | 23:00
 Spice Garden        | 23:00
 Sol y Mar           | 23:00
 Mango Bay Caribbean | 22:30
(8 rows)
```

Exercise 2 — Filtering with Containment and Key Existence

2.1 — Containment: `@>`

The containment operator checks whether the left JSONB document contains all the key-value pairs in the right document. It is the most common way to filter on JSONB values.

Find all waterfront businesses:

```
SELECT name, neighbourhood
FROM   businesses
WHERE  details @> '{"tags": ["waterfront"]}'
ORDER  BY neighbourhood, name;
```

```
          name           |  neighbourhood
-------------------------+-----------------
 Anchor & Oar Tavern     | Harbour District
 Harbour Inn             | Harbour District
 Harbour View Theater    | Harbour District
 Mariners Rest B&B       | Harbour District
 Portsmith Fish Market   | Harbour District
 Saltbox Gallery         | Harbour District
 The Gilded Clam         | Harbour District
 Tidal Wave Surf Shop    | Harbour District
 Old Brewery Tap         | Industrial Port
 The Rusty Anchor        | Industrial Port
(10 rows)
```

> **Why this works:** The `@>` operator checks whether the tags *array* on the left contains the tags array `["waterfront"]` on the right — array containment is supported natively.

2.2 — Key existence: `?`

The `?` operator tests whether a key exists in a JSONB object (or a value exists in a JSONB array). Unlike `@>`, it does not check the value — only presence.

Find all businesses that publish social media handles:

```
        SELECT name,
               details -> 'social' AS social_links
        FROM   businesses
        WHERE  details ? 'social'
        ORDER  BY name;
```

```
          name          |
social_links
------------------------
+-----------------------------------------------------------
 Bella Napoli           | {"instagram": "@bellanapoli_portsmith"}
 Le Petit Bistro        | {"instagram": "@lepetitbistro"}
 Quarter Note Jazz Club | {"instagram": "@quarternote_portsmith"}
 Spice Garden           | {"instagram": "@spicegarden_portsmith"}
 The Gilded Clam        | {"facebook": "thegildedclam",
"instagram": "@gildedclam"}
 The Riverside Vegan    | {"instagram": "@riverside_vegan"}
(6 rows)
```

2.3 — Any-key and all-key existence: `?|` and `?&`

`?|` returns true if *at least one* of the listed keys exists. `?&` returns true only if *all* listed keys exist.

```
        -- Businesses that offer either delivery OR takeaway
        SELECT name
        FROM   businesses
        WHERE  details ?| ARRAY['delivery', 'takeaway']
        ORDER  BY name;

        -- Businesses that offer BOTH delivery AND takeaway
        SELECT name
        FROM   businesses
        WHERE  details ?& ARRAY['delivery', 'takeaway']
        ORDER  BY name;
```

Run both and note the difference in result count. The `?&` set is a strict subset of the `?|` set.

Exercise 3 — Missing Keys vs. NULL Values

JSONB has two distinct ways to represent "no value": a JSON `null` and a **missing key**. They behave differently in queries.

In our data, a business closed on Monday can be represented two ways:

- `"mon": null` — the key exists, the value is JSON null
- the `mon` key is absent entirely

The seed data uses `"mon": null` for closures, so we can observe this.

3.1 — The difference matters for `?`

```
-- ? checks key EXISTENCE, not value
SELECT name
FROM   businesses
WHERE  (details -> 'hours') ? 'sun'   -- key exists (even if null)
ORDER  BY name;
```

Run it. You should get **43 rows** — every business that uses a day-keyed hours object. The five accommodation businesses are excluded because their hours look like `{"reception": "07:00-23:00"}` and have no `sun` key.

Now try what seems like a natural follow-up:

```
--   This does NOT do what you might expect
SELECT name
FROM   businesses
WHERE  (details -> 'hours' -> 'sun') IS NOT NULL
ORDER  BY name;
```

Run it. You still get **43 rows** — identical to the first query.

Why? This is one of JSONB's most important gotchas:

> **JSONB `null` is not SQL `NULL`.**

When the seed script stores `"sun": None` in Python, `json.dumps` produces `"sun": null` — a JSON null *value* under an existing key. When PostgreSQL evaluates `details -> 'hours' -> 'sun'` on such a row, it returns the JSONB value `null`. That is a real value — not the absence of a value — so `IS NOT NULL` evaluates to `TRUE`.

SQL `NULL` only appears when the key itself is **missing** from the object. That is what happens for the five accommodation businesses: `details -> 'hours'` returns `{"reception": "..."}`, and then `-> 'sun'` on that object finds no such key and returns SQL `NULL`, so `IS NOT NULL` is `FALSE`.

To correctly distinguish the three states, use `jsonb_typeof()`:

State	`details -> 'hours' -> 'sun'`	`jsonb_typeof(...)`	`IS NOT NULL`
Key missing (accommodation)	SQL `NULL`	SQL `NULL`	`FALSE`
Key present, closed (`null`)	JSONB `null`	`'null'`	**`TRUE` ← gotcha**
Key present, open (object)	JSONB object	`'object'`	`TRUE`

The correct query for "businesses actually open on Sunday":

```
SELECT name
FROM   businesses
WHERE  jsonb_typeof(details -> 'hours' -> 'sun') = 'object'
ORDER  BY name;
```

This returns **25 rows** — only the businesses with a real hours object for Sunday. `jsonb_typeof` returns `'object'` only for JSON objects, `'null'` for JSON null, and SQL `NULL` for a missing key (which then fails the `=` test).

> **Rule of thumb:** Never use `IS NOT NULL` to test whether a JSONB navigation result is a "real" value. Use `jsonb_typeof(...)` to check the actual JSON type, or navigate one level deeper (e.g., check `details #>> '{hours,sun,open}'`) — the `#>>` operator converts JSON null to SQL NULL, so a deeper path naturally filters it out.

3.2 — Handling accommodation differently

Hotels and hostels store hours as a single `"reception": "HH:MM-HH:MM"` string rather than a day-keyed object. This means the `? 'sun'` test returns false for them even though they may be open 24/7. Write a query that finds all businesses that are either:

- Open on Sunday (day-keyed hours with a non-null `sun`), **or**
- An accommodation with 24/7 reception

```
SELECT name, neighbourhood,
       CASE
           WHEN jsonb_typeof(details -> 'hours' -> 'sun') =
'object'
               THEN (details #>> '{hours,sun,open}') || '–' ||
                    (details #>> '{hours,sun,close}')
           WHEN details #>> '{hours,reception}' = '24/7'
               THEN '24/7'
           ELSE details #>> '{hours,reception}'
```

```
        END AS sunday_availability
FROM   businesses
WHERE  jsonb_typeof(details -> 'hours' -> 'sun') = 'object'
   OR  (details -> 'hours') ? 'reception'
ORDER  BY neighbourhood, name;
```

Exercise 4 — GIN Indexes

Without an index, every JSONB query is a sequential scan — PostgreSQL reads every row and evaluates the expression. For a 48-row toy dataset that is unnoticeable, but with millions of rows it becomes a bottleneck.

A **GIN** (Generalised Inverted Index) index over a JSONB column indexes every key and value inside every document. It accelerates `@>`, `?`, `?|`, and `?&` queries.

4.1 — Observe the plan before indexing

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT name
FROM   businesses
WHERE  details @> '{"tags": ["waterfront"]}';
```

With 48 rows you will see `Seq Scan` (sequential scan, i.e. reading every row) — that is expected. On a larger table this would say `Seq Scan` with a high cost estimate.

4.2 — Create the GIN index

```
CREATE INDEX idx_businesses_details_gin
    ON businesses
    USING GIN (details);
```

4.3 — Observe the plan after indexing

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT name
FROM   businesses
WHERE  details @> '{"tags": ["waterfront"]}';
```

On a small table PostgreSQL may still choose a sequential scan (the planner knows the table fits in memory). Force it to use the index to see the mechanism:

```
SET enable_seqscan = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT name
```

```
FROM   businesses
WHERE  details @> '{"tags": ["waterfront"]}';

SET enable_seqscan = on;   -- always restore this
```

You should now see `Bitmap Index Scan on idx_businesses_details_gin` in the plan. The GIN index will pay for itself in production as the table grows.

> **What GIN indexes:** The default `jsonb_ops` operator class indexes every key and value in the document, supporting `@>`, `?`, `?|`, `?&`. A `jsonb_path_ops` class indexes only values (not keys), producing a smaller index that is faster for `@>` but cannot accelerate `?` queries. Use `jsonb_path_ops` when you only ever query by value containment and index size matters.

```
-- Alternative: value-only index (smaller, faster @>, no ? support)
CREATE INDEX idx_businesses_details_pathops
    ON businesses
    USING GIN (details jsonb_path_ops);
```

Exercise 5 — Updating Documents in Place

JSONB columns are immutable at the document level — you cannot change a single key without rewriting the whole value. PostgreSQL gives you two tools to do that rewrite concisely.

5.1 — `jsonb_set`: replace or add one value

The Lighthouse Bookshop just received a flood of five-star reviews. Update its rating to 5.0:

```
UPDATE businesses
SET    details = jsonb_set(details, '{rating}', '5.0')
WHERE  name = 'Lighthouse Bookshop';
```

Verify:

```
SELECT name, details ->> 'rating' AS rating
FROM   businesses
WHERE  name = 'Lighthouse Bookshop';
```

`jsonb_set(target, path, new_value)` — the path is a text array of keys. If the key already exists it is replaced; if it does not exist it is added (by default — there is a fourth parameter to suppress creation).

5.2 — `jsonb_set` on a nested key

The Gilded Clam is now closing an hour later on Sundays:

```
        UPDATE businesses
        SET    details = jsonb_set(
                             details,
                             '{hours,sun,close}',
                             '"21:00"'     -- note: a JSON string must be
double-quoted
                         )
        WHERE  name = 'The Gilded Clam';

        SELECT name,
               details #>> '{hours,sun,close}' AS new_sunday_close
        FROM   businesses
        WHERE  name = 'The Gilded Clam';
```

5.3 — `||` merge operator: add or overwrite multiple keys at once

The `||` operator merges two JSONB objects, with the right side winning on conflicts. Use it to add a `verified` flag and update `review_count` in a single statement:

```
        UPDATE businesses
        SET    details = details || '{"verified": true, "review_count":
999}'
        WHERE  name = 'Anchor & Oar Tavern';

        SELECT name,
               details ->> 'verified'     AS verified,
               details ->> 'review_count' AS reviews
        FROM   businesses
        WHERE  name = 'Anchor & Oar Tavern';
```

> **Caution:** `||` does a *shallow* merge. If `details` contains a nested object like `hours` and you merge another object that also has `hours`, the entire `hours` value is replaced — not deep-merged. Use `jsonb_set` for targeted nested updates.

Exercise 6 — Expanding Arrays with `jsonb_array_elements`

The `tags` key in most business documents is a JSON array. To query across tags — for example, to find the most common ones across the whole directory — you need to expand the array into individual rows.

6.1 — Expand one business's tags

```
        SELECT name,
               jsonb_array_elements_text(details -> 'tags') AS tag
```

```
        FROM   businesses
        WHERE  name = 'The Gilded Clam';
```

```
       name       |      tag
------------------+---------------
 The Gilded Clam  | waterfront
 The Gilded Clam  | seafood
 The Gilded Clam  | romantic
 The Gilded Clam  | outdoor_seating
(4 rows)
```

> `jsonb_array_elements_text` is a set-returning function: each element of the array becomes its own row. The result has more rows than the input.

6.2 — Count tags across all businesses

```
        SELECT tag, COUNT(*) AS occurrences
        FROM   businesses,
               jsonb_array_elements_text(details -> 'tags') AS tag
        GROUP  BY tag
        ORDER  BY occurrences DESC
        LIMIT  15;
```

```
       tag        | occurrences
------------------+-------------
 waterfront       |          10
 takeaway         |           5
 family_friendly  |           4
 family_owned     |           4
 organic          |           3
 ...
(15 rows)
```

Your exact numbers will differ — this shows the query pattern. The comma between `businesses` and the `jsonb_array_elements_text(...)` call is a **lateral join** (PostgreSQL expands the function for each input row automatically when used in the `FROM` clause this way).

6.3 — Only businesses with multiple specific tags

Find restaurants that are both `vegan_options` and `takeaway`:

```
        SELECT name, details ->> 'cuisine' AS cuisine
        FROM   businesses
        WHERE  details @> '{"tags": ["vegan_options"]}'
          AND  details @> '{"tags": ["takeaway"]}';
```

Because each `@>` call is independent, both conditions must hold. This is more efficient than expanding the array when the GIN index is in place.

Exercise 7 — Path Queries with `jsonb_path_query`

The `@?` operator and `jsonb_path_query()` function implement **SQL/JSON path language** — a mini-query language for navigating and filtering within a JSONB document. It is more expressive than chained arrow operators for conditional navigation.

7.1 — Simple path existence

Find businesses where the social links include Instagram:

```
        SELECT name
        FROM   businesses
        WHERE  details @? '$.social.instagram';
```

`$.social.instagram` means: start at the root (`$`), descend to `social`, then to `instagram`. `@?` returns true if the path resolves to at least one value.

7.2 — Filter on a nested value

Find restaurants with a rating above 4.5:

```
        SELECT name,
               details ->> 'rating' AS rating
        FROM   businesses
        WHERE  details @? '$ ? (@.category == "restaurant" && @.rating >
4.5)';
```

```
         name          | rating
-----------------------+--------
 Bella Napoli          | 4.6
 Le Petit Bistro       | 4.7
 River Bend Bakery     | 4.8
 Spice Garden          | 4.6
 The Gilded Clam       | 4.5
(5 rows)
```

The `? (filter)` syntax inside a path expression is equivalent to a `WHERE` clause inside the document.

7.3 — Find businesses open late on Friday

Businesses that close at or after 21:00 on Friday (using string comparison, which works correctly for 24-hour `HH:MM` strings):

```
        SELECT name,
               details #>> '{hours,fri,close}' AS friday_close
        FROM   businesses
```

```
        WHERE  details @? '$.hours.fri.close ? (@ >= "21:00")'
        ORDER  BY friday_close DESC, name;
```

```
           name           | friday_close
--------------------------+--------------
 Bella Napoli             | 23:00
 Dragon Palace            | 23:00
 Harbour View Theater     | 23:00
 The Hungry Scholar       | 23:00
 Spice Garden             | 23:00
 Sol y Mar                | 23:00
 Mango Bay Caribbean      | 22:30
 The Gilded Clam          | 22:30
 Thai Orchid              | 22:00
 The Riverside Vegan      | 22:00
 Northgate Grocers        | 21:00
 Lotus Spa & Wellness     | 20:00
(12 rows)
```

> **Limitation to notice:** Businesses that close *after midnight* — Anchor & Oar, The Clocktower Pub, The Rusty Anchor, and others — show a `close` of `"01:00"` or `"02:00"`. Lexicographically, `"02:00" < "21:00"`, so these businesses are *excluded* from the above results even though they are open very late. This is a design trade-off of storing times as plain strings. One solution is to store closing times past midnight as `"25:00"`, `"26:00"`, etc. — an unusual but practical convention for 24-hour time arithmetic.

7.4 — Extract values with `jsonb_path_query`

`@?` is a boolean test. `jsonb_path_query()` returns the actual matched values:

```
        SELECT name,
               jsonb_path_query(details, '$.hours.fri.close') AS
friday_close_json
        FROM   businesses
        WHERE  details @? '$.hours.fri'
        ORDER  BY name
        LIMIT  8;
```

The returned values are JSONB (note the quotes around the time strings). Use `jsonb_path_query_first(...) #>> '{}'` to extract a single scalar as text without the surrounding quotes.

Summary — What You Should Now Know

You have worked through the core JSONB toolkit. Here is what each operator and function you used actually does:

Tool	What it does
`-> 'key'`	Navigate one level; returns **JSONB**
`->> 'key'`	Navigate one level; returns **text**
`#>> '{a,b,c}'`	Navigate a path; returns **text**; JSON null becomes SQL NULL
`@> '{"k":"v"}'`	Containment test; accelerated by GIN
`? 'key'`	Key exists test (regardless of value); accelerated by GIN
`?|` / `?&`	Any-key / all-keys exist; accelerated by GIN
`jsonb_typeof(val)`	Returns `'object'`, `'array'`, `'string'`, `'number'`, `'boolean'`, or `'null'`; SQL NULL for a missing key
`jsonb_set(col, path, val)`	Replace or insert a nested value
`col || '{"k":"v"}'`	Shallow-merge a document
`jsonb_array_elements_text(col)`	Expand a JSON array into rows
`@? 'path'`	SQL/JSON path existence test
`jsonb_path_query(col, 'path')`	SQL/JSON path — return matched values

> **Remember:** JSONB `null` ≠ SQL `NULL`. Use `jsonb_typeof()` — not `IS NOT NULL` — when you need to distinguish a missing key from a key that exists but holds a JSON null value.

The key design insight from this chapter is that JSONB lets you keep a relational spine — the columns your queries always need (`id`, `name`, `neighbourhood`) — while storing everything else in a document whose shape can vary row by row. The GIN index means you do not pay a query performance penalty for that flexibility.

In the next chapter you will add point geometry to the `businesses` table and use PostGIS to answer spatial questions: which businesses are within 500 metres of the harbour, and which neighbourhood does each one belong to.

Going further: PostgreSQL 14+ supports the `jsonpath` type natively. The `jsonb_path_query_array` and `jsonb_path_query_first` variants are useful for pagination and single-value extraction. For write-heavy workloads,

profile whether `JSONB` or a computed stored column (Chapter 16) gives better `INSERT`/`UPDATE` throughput on your hardware.